Introduction to Computer

Professor: Yu-Guang Chen

Final Project

A CNN Accelerator Simulator

Department:EE4B

Source A: ember-otter-3c61

Source B: cobalt-heron-8b24

I. 程式架構

```
/* ===== Read image ===== */
vector<vector<float>> img(28, vector<float>(28));
ifstream fin(input_addr + "img_label.txt");
for (int i = 0; i < 28; ++i)
    for (int j = 0; j < 28; ++j)
    {
        char c;
        fin >> img[i][j];
        if (j != 27) fin >> c;
    }
fin.close();
```

程式負責將輸入影像由文字檔正確轉換為二維矩陣格式，並處理逗號分隔的檔案結構。

1.影像資料結構配置

本程式使用二維 vector 宣告一個大小為 28×28 的浮點數矩陣 img，用以儲存單張灰階影像的像素值。此資料結構對應後續卷積層所需的輸入格式，方便以 [row][column] 方式存取像素資料。

2.讀取影像檔案 (img_label.txt)

透過 ifstream 開啟影像檔案，檔案內容為一張 28×28 的影像，每一列包含 28 個數值，並以逗號作為分隔符號。程式以 for 迴圈逐一讀取每個像素值，確保影像資料依照原始排列順序正確存入矩陣。

3.處理逗號分隔格式

由於影像檔案中的數值以逗號分隔，若直接使用 >> 讀取，逗號可能導致讀取錯誤。因此程式額外宣告一個 char c，在每一列中讀取並捨棄逗號字元，避免逗號殘留於輸入串流中而影響後續資料解析。

4.避免輸入卡住的設計

在每一列的最後一個數值（j == 27）不再讀取逗號，確保輸入串流正確對齊換行符號，避免發生讀取錯位或卡住的情況。

5.關閉檔案串流

完成影像讀取後，立即呼叫 fin.close() 關閉檔案，以釋放系統資源並確保後續檔案操作不受影響。

```
/* ===== Quantize ===== */
vector<vector<vector<float>>> img_q(1,
                                    vector<vector<float>>(28, vector<float>(28))
                                   );
double err = 0;
for (int i = 0; i < 28; ++i)
    for (int j = 0; j < 28; ++j)
    {
        unsigned char q = fp32_to_fp8_e4m3(img[i][j]);
        float xhat = fp8_to_fp32_e4m3(q);
        img_q[0][i][j] = xhat;
        err += fabs(img[i][j] - xhat) / max(1e-12, (double)fabs(img[i][j]));

    }
float avg_relative_error = err / (28 * 28) * 100.0f;
```

程式負責模擬低精度 FP8（E4M3）量化對輸入影像的影響，並透過反量化與誤差分析，驗證量化後資料仍可作為後續卷積運算的有效輸入，同時量化誤差亦能被明確評估。

1.量化後影像資料結構設計

為配合後續卷積層的輸入格式，本程式將量化後的影像宣告為三維 vector，其維度為

[Cin = 1][28][28]。其中第一維代表通道數，後兩維則對應影像的空間尺寸。此設計可使量化後影像直接作為 conv1 的輸入，避免後續額外的資料重排。

2.FP32 到 FP8 量化流程

程式對原始影像中每一個像素值進行逐點量化，首先使用 fp32_to_fp8_e4m3() 函式，將 32-bit 浮點數轉換為 FP8 格式。該格式包含 1-bit sign、4-bit exponent 及 3-bit mantissa，並以 bias = 7 表示指數範圍，用以模擬低精度運算環境。

3.FP8 到 FP32 還原

為了讓後續卷積計算仍可在 CPU 上以浮點數進行，量化後的 FP8 資料會立即透過 fp8_to_fp32_e4m3() 函式還原為 FP32。

4.量化後影像儲存

還原後的數值 xhat 會存入 img_q[0][i][j]，作為後續卷積層的實際輸入資料。此時的 img_q 已完整反映 FP8 量化與反量化對影像所造成的影響。

5.平均相對誤差計算

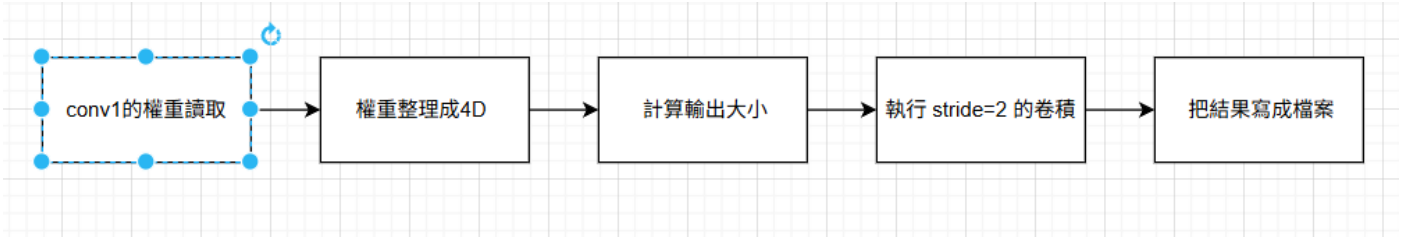
為評估量化對數值精度的影響，程式在量化過程中同步計算每一像素的相對誤差，定義為：

$$\frac{|x - \hat{x}|}{|x|}$$

其中為避免原始像素值為零時發生除以零的問題，分母以 max(1e-12, |x|) 作為保護。最終將所有像素的相對誤差取平均，並乘以 100%，得到整張影像的平均相對誤差。

6.量化設定

本實作依照專題規格說明，未支援 subnormal number，當數值小於最小可表示的正常數（2^{-6}）時，將直接 flush 為零。相關行為已於 FP8 轉換函式中妥善處理，確保與官方 checker 的量化邏輯一致。




```
/* ===== conv1 ===== */
int n1;
fin.open(input_addr + "conv1_weight.txt");
auto c1raw = pass_weight(fin, n1, 2);
fin.close();
```

程式負責將第一層卷積所需的權重由檔案讀入，並轉換為程式內部可直接使用的三維矩陣格式，為後續 conv1 卷積運算提供正確且彈性的權重資料來源。

1.conv1 權重檔案讀取
透過 ifstream 開啟第一層卷積所使用的權重檔案 conv1_weight.txt。該檔案中存放的是 conv1 層所有卷積 kernel 的權重數值，並以文字格式、逗號分隔方式儲存。

2.權重數量與 kernel 大小推導
函式 pass_weight() 負責解析權重檔內容，並將所有數值讀入後重新整理成適合程式使用的資料結構。由於 conv1 的輸出通道數為 2，且輸入影像僅有單一通道，因此在呼叫 pass_weight() 時，第三個參數設定為 2，代表共有兩片卷積 kernel。

3.自動推回 kernel 尺寸
在 pass_weight() 函式內，會依據權重檔案的總行數與 kernel 數量，計算每一片 kernel 的邊長 n1。此方式使程式能自動支援不同大小的卷積 kernel，而不需在程式中硬編碼 kernel 尺寸，提升程式彈性。

4.權重資料結構
pass_weight() 的回傳值 c1raw 為三維資料結構，其維度為：

c1raw[2][n1][n1]

其中第一維對應 conv1 的輸出通道數，每一個輸出通道皆有一片 $n1 \times n1$ 的卷積 kernel。由於 conv1 僅有單一輸入通道，因此權重不需再區分輸入通道索引。

5.檔案關閉
完成權重讀取後，程式立即關閉檔案串流，確保系統資源被正確釋放，並避免後續檔案操作受到影響。

```
vector<vector<vector<vector<float>>>> conv1_w(2,
        vector<vector<vector<float>>>(1,
                                     vector<vector<float>>(n1, vector<float>(n1))
                                    )
                                          );
for (int oc = 0; oc < 2; ++oc)
    conv1_w[oc][0] = c1raw[oc];
```

程式將 conv1 權重由 c1raw[2][n1][n1] 轉換為通用的 4D 格式 conv1_w[2][1][n1][n1]，使後續卷積運算能以一致的資料結構進行。

雖然 pass_weight() 讀進來的 c1raw 已經包含 conv1 的兩片 kernel，格式為 c1raw[2][n1][n1]，但後續卷積運算函式 conv_layer() 的權重參數設計為通用形式：

weight[Cout][Cin][K][K]

因此在呼叫 conv_layer() 前，需要先將 conv1 權重整理為 4 維結構，以符合統一的卷積介面，讓同一個 conv_layer() 能同時支援 conv1 與 conv2。

1.宣告 conv1 的 4D 權重容器
程式宣告 conv1_w 為四維 vector，其維度含意如下：

- 第一維 Cout = 2：conv1 的輸出通道數為 2，因此有兩組輸出特徵圖，每組對應一片 kernel
- 第二維 Cin = 1：輸入影像為灰階單通道，因此輸入通道數為 1
- 第三、四維 n1 × n1：每片 kernel 的空間尺寸

整體權重形狀可寫為：conv1_w[2][1][n1][n1]

2.將 2D kernel 塞入 4D 權重結構
接著透過迴圈將 c1raw 讀到的每一片 n1×n1 kernel 指派到 conv1_w 的對應位置：

conv1_w[oc][0] = c1raw[oc];

其中：

- oc 表示輸出通道索引（0 或 1）
- [0] 表示唯一的輸入通道（Cin=1）
- c1raw[oc] 即為對應的 n1×n1 kernel 權重矩陣
  透過此轉換後，conv1 權重即可用與 conv2 相同的介面傳入 conv_layer()，提高程式模組化與可維護性。

```
    int OUT1 = (28 - n1) / 2 + 1;
    vector<vector<vector<float>>> conv1_out(
        2, vector<vector<float>>(OUT1, vector<float>(OUT1))
    );

    conv_layer(img_q, conv1_out, conv1_w, conv1_b, 2);
```

程式依據卷積參數推導輸出尺寸，並完成第一層卷積運算，將量化後影像轉換為多通道特徵圖，作為後續卷積層的輸入。

1.Conv1 輸出尺寸

第一層卷積採用 stride = 2 且不使用 padding，因此輸出特徵圖的空間尺寸需依據標準卷積公式計算。對於輸入大小為 28×28、kernel 大小為 n1×n1 的情況，輸出尺寸 OUT1 定義如下：

$$OUT1 = \frac{28 - n1}{2} + 1$$

此公式反映卷積視窗每次以 2 個像素為步長滑動，並在不超出輸入邊界的前提下產生輸出。

2.Conv1 輸出特徵圖配置

根據 conv1 的輸出通道數（Cout = 2），程式宣告三維向量 conv1_out 作為輸出容器，其維度為：

conv1_out[2][OUT1][OUT1]

第一維對應兩個輸出通道，其餘兩維則對應空間位置，用以儲存卷積後的特徵圖結果。

3.執行 Conv1 卷積運算

透過呼叫 conv_layer() 函式，實際執行第一層卷積運算。該函式接收量化後影像 img_q 作為輸入、整理後的 4D 權重 conv1_w、對應的 bias conv1_b，並以 stride = 2 進行卷積。

```
    int Hout = (H - K) / STRIDE + 1;
    int Wout = (W - K) / STRIDE + 1;

    for (int oc = 0; oc < Cout; ++oc)
    {
        reset_buffer_and_matrix();

        for (int orow = 0; orow < Hout; ++orow)
        {
            for (int ocol = 0; ocol < Wout; ++ocol)
            {
                /* ===== CPU correctness ===== */
                float sum = bias[oc];
                int ir0 = orow * STRIDE;
                int ic0 = ocol * STRIDE;

                for (int ic = 0; ic < Cin; ++ic)
                    for (int kr = 0; kr < K; ++kr)
                        for (int kc = 0; kc < K; ++kc)
                            sum += input[ic][ir0 + kr][ic0 + kc]
                                 * weight[oc][ic][kr][kc];

                output[oc][orow][ocol] = sum;


                /* ===== Hardware energy ===== */
                for (int ic = 0; ic < Cin; ++ic)
                {
                    float w3[3][3] = {0};
                    for (int a = 0; a < 3; ++a)
                        for (int b = 0; b < 3; ++b)
                            if (a < K && b < K)
                                w3[a][b] = weight[oc][ic][a][b];
                    set_weight(w3);

                    for (int kc = 0; kc < K; ++kc)
                    {
                        float new_data[3];
                        new_data[0] = input[ic][ir0 + 0][ic0 + kc];
                        new_data[1] = input[ic][ir0 + 1][ic0 + kc];
                        new_data[2] = input[ic][ir0 + 2][ic0 + kc];

                        if (ir0 < 2)
                            input_selection(false, 0, false, 0, new_data);
                        else
                            input_selection(true, 1, true, 1, new_data);
                    }
                }
            }
        }
    }
}
```

CPU correctness 部分確保卷積輸出與理論計算一致，而 hardware energy 部分則模擬實際加速器中權重載入、資料輸入與資料重用行為，並累積對應的能耗與運算步數。

1.卷積輸出尺寸計算

在進行卷積運算前，程式會依據輸入特徵圖大小、kernel 尺寸 K 以及 stride 設定，計算輸出特徵圖的空間尺寸。由於本系統未使用 padding，輸出大小可由下式推得：

$$H_{out} = \frac{H - K}{\text{STRIDE}} + 1, W_{out} = \frac{W - K}{\text{STRIDE}} + 1$$

此公式確保卷積視窗在不超出輸入邊界的情況下，以固定步長滑動。

2.以輸出通道為單位進行卷積

最外層迴圈以輸出通道 oc 為單位進行運算。每進入一個新的輸出通道時，會呼叫 reset_buffer_and_matrix()，重置硬體模擬中使用的 buffer 與暫存矩陣，確保不同輸出通道的能耗與步數計算不互相干擾。

3.卷積視窗起始位置

對於每一個輸出位置，程式會依據 stride 設定計算對應的輸入起始座標 (ir0, ic0)，表示卷積視窗左上角在輸入特徵圖中的位置。

4.卷積數值運算

對每個輸出通道 oc，會累加所有輸入通道 ic 與對應 kernel 權重的乘積，並加上 bias。其數學形式可表示為：

$$output[oc][orow][ocol = bias[oc] + \sum_{ic} \sum_{kr,kc} input[ic][ir0 + kr][ic0 + kc] \cdot weight[oc][ic][kr][kc]$$

5.權重載入

程式將實際 kernel 權重嵌入至固定大小的 3×3 矩陣 w3 中。若 kernel 尺寸小於 3×3，則其餘位置補零。此設計反映硬體加速器中常見的固定運算陣列大小限制。透過 set_weight() 將權重送入模擬硬體，並累積相應的能耗與步數。

6.輸入資料串流模擬

在每一個 kernel column 上，從輸入特徵圖中取出對應的三個資料，形成一次送入運算陣列的輸入資料向量。

8.輸入選擇

根據目前卷積視窗位置，程式選擇不同的 input_selection() 參數組合，以模擬硬體中 line buffer 與資料重用行為。此設計用以評估資料流策略對能耗與 step count 的影響。

Conv1 直接對原始影像進行卷積，用以萃取低階特徵，Conv2 則在此基礎上整合多個特徵通道，形成較高階的特徵表示，供後續全連接層使用

Layer	**Cin**	**Cout**	**Kernel**	**Stride**	**Output size**
Input	1				28×28
Conv1	1	2	n1×n1	2	OUT1×OUT1
Conv2	2	4	n2×n2	2	OUT2×OUT2
FC	4×OUT2×OUT2	10			10

```
// ---- Read FC weights & bias ----
    vector<float> fc_weight_flat =
        read_csv_floats(input_addr + "fc_weight.txt");    // size = 10 * FC_IN
    vector<float> fc_bias =
        read_csv_floats(input_addr + "fc_bias.txt");      // size = 10

// 安全檢查（不影響 judge，但避免爆掉）
    if ((int)fc_weight_flat.size() != FC_OUT * FC_IN)
    {
        cerr << "[ERROR] FC weight size mismatch\n";
    }

// ---- FC computation ----
    vector<float> fc_out(FC_OUT, 0.0f);

    for (int o = 0; o < FC_OUT; ++o)
    {
        float sum = fc_bias[o];
        int idx = 0;   // implicit flatten index

        // ★ Channel-major + row-major（跟教材圖一致）
        for (int c = 0; c < 4; ++c)
            for (int i = 0; i < OUT2; ++i)
                for (int j = 0; j < OUT2; ++j)
                {
                    float x = conv2_out[c][i][j];
                    float w = fc_weight_flat[o * FC_IN + idx];
                    sum += w * x;
                    idx++;
                }

        fc_out[o] = sum;
    }

    int final_pred = 0;
    for (int i = 1; i < 10; ++i) if (fc_out[i] > fc_out[final_pred]) final_pred= i;
```

全連接層將第二層卷積輸出的多通道特徵圖依固定順序攤平成一維向量，並與對應權重進行內積運算，最終產生十個分類分數。

1.全連接層權重與偏置讀取

透過 read_csv_floats() 函式，將全連接層（FC layer）所需的權重與偏置自檔案中讀入。由於全連接層共有 10 個輸出節點，分別對應數字 0 至 9，因此權重檔案 fc_weight.txt 的總長度應為 $10 \times FC_IN$，而 fc_bias.txt 則包含 10 個偏置值。

2.權重尺寸檢查(不影響)

為避免因檔案格式錯誤或資料遺失導致程式於執行時發生未定義行為，本程式在讀取權重後進行尺寸檢查。若權重數量與預期的 $FC_OUT \times FC_IN$ 不一致，程式會輸出錯誤訊息。

3.全連接層輸出向量配置

宣告一個大小為 FC_OUT 的一維向量 fc_out，用以儲存全連接層的輸出結果。每一個元素代表對應分類的分數，尚未經過 softmax 正規化。

4.初始化單一輸出節點的累加值

對於每一個輸出節點 o，程式先將對應的 bias 值作為初始累加值，並使用 idx 作為攤平後輸入向量的索引，確保權重與輸入特徵能正確對應。

5. 特徵圖攤平與內積運算

將第二層卷積輸出的多通道特徵圖，依固定順序攤平成一維向量。攤平順序採用 channel-major，再 row-major 的方式，即先遍歷所有通道，再依序展開每個通道內的空間位置。此順

序需與權重訓練時的排列方式一致，否則將導致分類結果錯誤。

在攤平過程中，程式同時完成全連接層的內積運算，將每一個輸入特徵值與對應權重相乘後累加至 sum 中，其數學形式可表示為：

$$fc_out[o] = bias[o] + \sum_{k=0}^{FC_IN-1} W[o,k] \cdot x[k]$$

6.預測結果判定

全連接層完成後，程式透過比較 fc_out 中的 10 個分數，選取最大值對應的索引作為模型的最終預測結果。由於 softmax 不影響最大值的位置，因此本實作直接使用 argmax 判定分類結果，以簡化計算流程。

```
static inline float fp8_to_fp32_e4m3(unsigned char b)
{
    int sign = (b >> 7) & 1;
    int E = (b >> 3) & 0xF;
    int M = b & 0x7;
    if (E == 0 && M == 0) return 0.0f;

    float mant = 1.0f + (float)M / 8.0f;
    float x = ldexp(mant, E - 7);
    return sign ? -x : x;
}
```

此函式負責將一個以 FP8(E4M3) 格式儲存的 8-bit 無號整數，轉換回 IEEE 754 單精度浮點數。該反量化過程用於模擬低精度運算後，將資料還原至高精度格式以進行後續計算或結果分析。

1.位元欄位解析

FP8 格式由三個欄位組成：

- Sign bit（1 bit）：最高位元，用以表示數值正負。
- Exponent（4 bits）：指數欄位，採用 bias。
- Mantissa（3 bits）：尾數欄位，表示有效數字的小數部分。

程式透過位元運算將輸入的 8-bit 資料拆解為對應的三個欄位，為後續數值重建做準備。

2. 零值判斷

當 exponent 與 mantissa 皆為 0 時，表示該數值為 ±0。由於不支援 subnormal number，此情況直接回傳 0.0。

3. 尾數重建

FP8(E4M3) 採用隱含的 leading 1 normalized representation。因此在反量化時，尾數需以：

$$\text{mantissa} = 1 + \frac{M}{2^3}$$

的形式重建，確保數值能正確反映原始浮點表示。

4. 指數還原

指數欄位使用 bias = 7 的偏移編碼方式。反量化時，需將實際指數還原為：

$$2^{(E-\text{bias})}$$

此處透過 ldexp() 函式同時完成乘以 2 的冪次與 mantissa 的組合，能避免手動計算時的誤差。

5. 正負號還原

最後根據 sign bit 決定數值正負，完成 FP8 至 FP32 的反量化轉換。

```
static inline unsigned char fp32_to_fp8_e4m3(float x)
{
    const int BIAS = 7;
    if (!isfinite(x)) return 0x77;
    if (x == 0.0f) return 0;

    int sign = (x < 0.0f);
    float ax = fabs(x);

    int e2;
    float m = frexp(ax, &e2);
    float f2 = m * 2.0f;
    int exp = e2 - 1;

    int E = exp + BIAS;
    if (E <= 0) return 0;
    if (E >= 15) return (unsigned char)((sign << 7) | (14 << 3) | 7);

    int mant = (int)round((f2 - 1.0f) * 8.0f);
    if (mant == 8)
    {
        mant = 0;
        E++;
        if (E >= 15) return (unsigned char)((sign << 7) | (14 << 3) | 7);
    }

    return (unsigned char)((sign << 7) | ((E & 0xF) << 3) | (mant & 0x7));
}
```

函式完成 FP8 的數值轉換邏輯，並依據官方規範處理 subnormal、特殊值與飽和情況，使量化結果能穩定用於後續卷積運算。

1.不支援 subnormal number

FP8(E4M3) 不支援 subnormal number。當輸入數值小於最小可表示的正常數（2^{-6}）時，程式會直接將其量化為 0。

2.NaN 與 Inf 的處理方式

若輸入為 NaN 或 Inf，程式會回傳固定的保留編碼 0x77，用以表示非有限數值。此處理方式確保特殊數值不會在後續運算中造成未定義行為。

3.數值過大時的飽和處理

當輸入數值超過 FP8 可表示的最大範圍時，量化結果會被限制在最大可表示的有限值，而非發生溢位。此作法可避免數值爆炸，並模擬實際硬體中常見的飽和行為。

4.四捨五入量化策略

在 mantissa 計算過程中，程式採用四捨五入方式進行量化，而非單純截斷。若四捨五入導致 mantissa 溢位，則透過進位調整 exponent，確保量化結果仍維持正規化格式。

```
vector<vector<vector<float>>> pass_weight(ifstream &fin, int &n, int sets)
{
    vector<float> raw;
    string line;
    int row_count = 0;

    while (getline(fin, line))
    {
        int start = 0;
        while (true)
        {
            int comma = (int)line.find(',', start);
            if (comma == (int)string::npos)
            {
                string token = line.substr(start);
                if (!token.empty()) raw.push_back(stof(token));
                break;
            }
            else
            {
                raw.push_back(stof(line.substr(start, comma - start)));
                start = comma + 1;
            }
        }
        row_count++;
    }

    n = row_count / sets;

    vector<vector<vector<float>>> weight(
        sets, vector<vector<float>>(n, vector<float>(n, 0.0f))
    );

    int idx = 0;
    for (int s = 0; s < sets; ++s)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                weight[s][i][j] = raw[idx++];

    return weight;
}
```

pass_weight() 會把一個權重檔讀成一維，再用 n =行數/sets 推回 kernel size，最後把 raw reshape 成 weight[sets][n][n] 回傳

II.測資

```
#################################################################
Running Checks for TEST01
For Kernel size = 3x3
#################################################################
Setting up input files from: /app/cases/Testcase01
Setup Success: All 7 files prepared.

================================================
Correctness Check
================================================
result_of_conv1.txt: PASS
result_of_conv2.txt: PASS
result_of_fc.txt: PASS

Relative Error after Quantization match: True
Result of the Prediction match: True

The correctness check succeeded. Starting the optimization check.

================================================
Optimization Check
================================================

--- Optimization Scoring Results ---

Energy Baseline: 192290.0
Actual Energy: 34630.0
Energy Score: 2.0

Step Baseline: 6408.0
Actual Step: 1878.0
Step Score: 2.0

--- Your optimize score of this testcase is 4.0 ---
```

```
#################################################################
Running Checks for TEST03
For Kernel size = 5x5
#################################################################
Setting up input files from: /app/cases/Testcase03
Setup Success: All 7 files prepared.

================================================
Correctness Check
================================================
result_of_conv1.txt: PASS
result_of_conv2.txt: PASS
result_of_fc.txt: PASS

Relative Error after Quantization match: True
Result of the Prediction match: True

The correctness check succeeded. Starting the optimization check.

================================================
Optimization Check
================================================

--- Optimization Scoring Results ---

Energy Baseline: 697160.0
Actual Energy: 37480.0
Energy Score: 2.0

Step Baseline: 23232.0
Actual Step: 2080.0
Step Score: 2.0

--- Your optimize score of this testcase is 4.0 ---
```

```
#################################################################
Running Checks for TEST02
For Kernel size = 7x7
#################################################################
Setting up input files from: /app/cases/Testcase02
Setup Success: All 7 files prepared.

================================================
Correctness Check
================================================
result_of_conv1.txt: PASS
result_of_conv2.txt: PASS
result_of_fc.txt: PASS

Relative Error after Quantization match: True
Result of the Prediction match: True

The correctness check succeeded. Starting the optimization check.

================================================
Optimization Check
================================================

--- Optimization Scoring Results ---

Energy Baseline: 946380.0
Actual Energy: 39370.0
Energy Score: 2.0

Step Baseline: 31536.0
Actual Step: 2198.0
Step Score: 2.0

--- Your optimize score of this testcase is 4.0 ---
```

```
************************************************

Completed checks for all test cases.
Final Correctness Score: 30 out of 30
Final Optimization Score: 12.0 out of 12

************************************************

Kernel Size 3*3: Energy = 34630.0, Step = 1878
Kernel Size 7*7: Energy = 39370.0, Step = 2198
Kernel Size 5*5: Energy = 37480.0, Step = 2080

************************************************
```

從測資結果可知，能耗與 step 數主要受 kernel size 影響，kernel 越大，所需的運算次數與資料存取次數越多，因此整體硬體能耗亦隨之提升。此外，由於本專題在 conv layer 中明確區分 CPU correctness 與 hardware energy 模擬流程，使得在確保數值正確的同時，也能反映不同卷積參數對硬體成本的影響。

III.困難

讓我最頭痛的其實不是寫某一個函式，而是整個流程中各種細節彼此牽連。很多時候程式可以順利編譯、也能跑完，但只要某個地方稍微不對，最後結果就會整個錯掉，而且錯誤來源還不容易馬上看出來。

首先是資料維度的推導。在算卷積輸出大小時，需要同時考慮輸入尺寸、kernel size 跟 stride，只要公式少一個括號或整數除法處理不一致，後面的 indexing 就會開始出問題。有時候程式不會直接 crash，而是悄悄用錯資料在算，等到最後 FC 層結果不對時，才發現其實是前面某一層的尺寸早就算歪了。

再來是權重檔案的格式假設。我在讀取權重時，是先把檔案內容全部讀成一維，再用行數去推回 kernel size，最後再 reshape 成多維矩陣。這種寫法雖然彈性高，但也很吃檔案格式，只要權重檔多一行、少一行，或行數沒辦法被 kernel 數量整除，整個 reshape 就會錯位，卷積結果也會跟著全部亂掉。這類錯誤很煩，因為程式看起來都正常，但結果完全對不上。

在 conv2 的權重重排 上我也卡了很久。因為 conv2 同時有多個 input channel 跟 output channel，權重檔是用比較扁平的方式給，要自己重新整理成 [Cout][Cin][K][K]。只要哪兩片 kernel 應該放在同一個 output channel 卻配錯了，數值就會明顯跑掉，但從輸出結果其實很難直接看出是哪一組權重出問題。

另外一個讓我踩雷的是 FC 層的 flatten 順序。就算前面的 conv1、conv2 都算對了，只要在攤平特徵圖時順序跟權重檔原本的順序不一致，FC 算出來的結果就會完全不合理。這種錯誤特別難抓，因為程式可以正常跑完，但分類結果就是一直不對，最後只能一個一個對照 flatten 的順序慢慢查。

在量化與硬體模擬的部分，也花了不少時間調整。FP8(E4M3) 有很多細節規定，像是 subnormal 要直接變成 0，如果沒有完全照官方規則實作，就算卷積流程寫對了，量化後的數值也會一開始就跟標準答案不一樣。再加上卷積函式裡同時有「CPU 算數值」跟「硬體能耗模擬」兩條路徑，有時候數值是對的，但硬體呼叫順序不對，還是會影響最後的 energy 跟 step。

最後是輸出格式的問題。像是換行位置、逗號、還有小數點怎麼截斷，這些看起來很小的細節，只要跟規定不一樣，checker 就不會過。為了這些格式問題，我也反覆改了好幾次，才全部對齊。

老實說，這個 PA 一開始做起來滿挫折的，因為很多錯誤都不是那種一眼就能看出來的錯。程式可以正常編譯、也能跑完，但結果就是不對，只能一層一層檢查。最後成功通過所有測資時，其實滿有成就感的，也讓我對 CNN 與硬體模擬更理解。